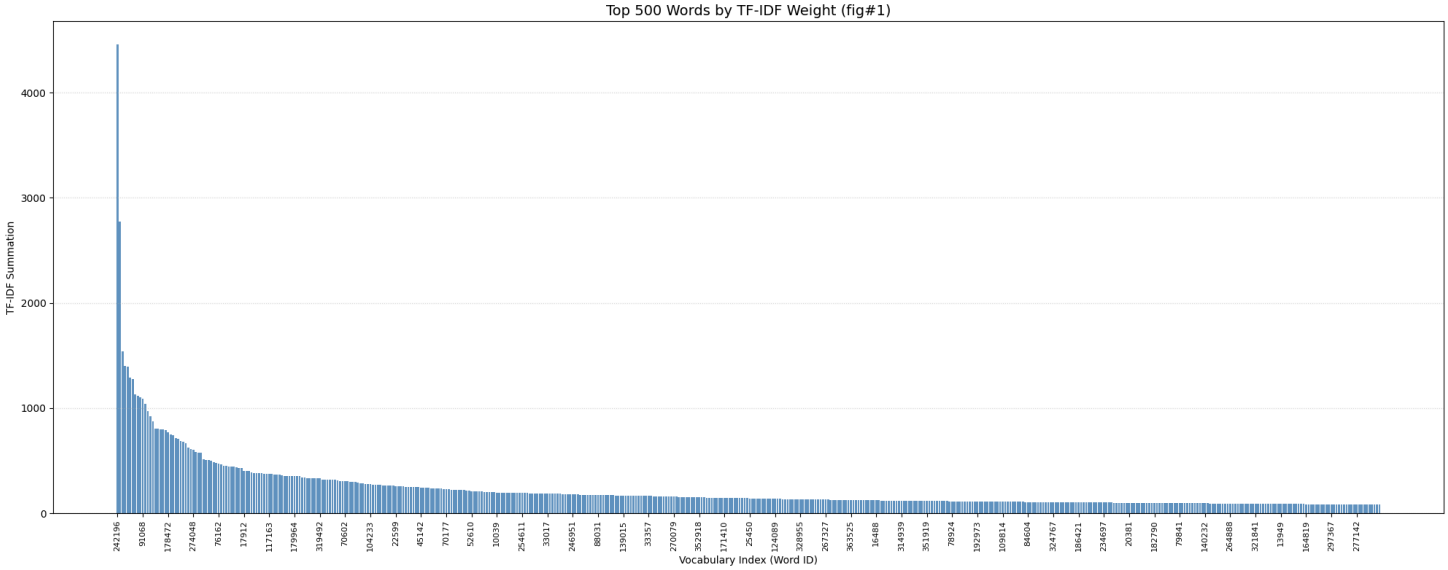
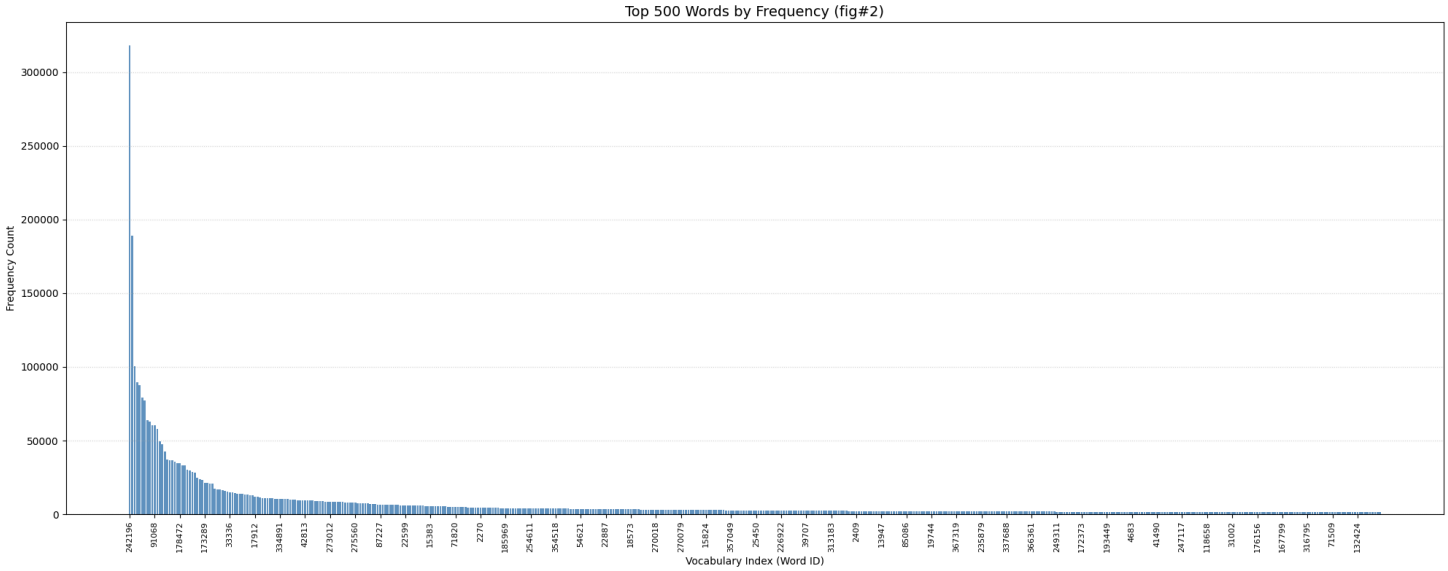
```python
import warnings
import logging
import requests
import jieba
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from typing import List, Tuple

# Environment Configuration
warnings.filterwarnings("ignore", category=SyntaxWarning)
logging.getLogger('matplotlib').setLevel(logging.ERROR)
jieba.setLogLevel(logging.ERROR)

def plot_results(data: List[Tuple[str, float]], title: str, ylabel: str,
    fig_num: int, word2idx: dict):
    """Generates analytical visualizations for token metrics."""
    # 【嚴格邏輯】：確保 TF 和 TF-IDF 使用完全相同的 Vectorizer 邏輯，字庫絕對一致。
    indices = [word2idx[word] for word, _ in data]
    metrics = [val for _, val in data]

    plt.figure(figsize=(20, 8))
    x_axis = np.arange(len(indices))
    plt.bar(x_axis, metrics, color='steelblue', alpha=0.85)

    plt.title(title, fontsize=14)
    plt.xlabel("Vocabulary Index (Word ID)", fontsize=10)
    plt.ylabel(ylabel, fontsize=10)
    plt.xticks(x_axis[::10], indices[::10], rotation=90, fontsize=8)
    plt.grid(axis='y', linestyle=':', alpha=0.5)
    plt.tight_layout()
    plt.savefig(f'fig#{fig_num}.png')
    plt.show()

if __name__ == "__main__":
    # Data Acquisition
    print("[Status] Fetching remote dataset...")
    url = 'https://github.com/cjwu/cjwu.github.io/raw/master/courses/nlp/
    hw1-dataset.txt'
    raw_data = requests.get(url).text.strip().split('\n')

    # Preprocessing: 10 lines per document
```

```python
docs: List[str] = [].join(raw_data[i : i + 10]) for i in range(0, len(raw_data), 1
print(f"[Status] Dataset initialized. Total documents: {len(docs)}")

# Pipeline: Word Segmentation
print("[Status] Executing tokenization...")
tokenized_docs = []
try:
    jieba.enable_parallel(8)
    for doc in docs:
        tokenized_docs.append(' '.join(jieba.lcut(doc)))
except Exception:
    for doc in docs:
        tokenized_docs.append(' '.join(jieba.lcut(doc)))
finally:
    jieba.disable_parallel()

# Pipeline: Feature Engineering
print("[Status] Computing vector weights...")

# 【嚴格邏輯】：TF-IDF 與 TF 必須使用完全相同的 Tokenizer 邏輯
# 1. 計算 TF-IDF
tfidf_vectorizer = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
tfidf_matrix = tfidf_vectorizer.fit_transform(tokenized_docs)

word2idx = tfidf_vectorizer.vocabulary_
feature_names = tfidf_vectorizer.get_feature_names_out()

# 2. 計算 TF
tf_vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
tf_matrix = tf_vectorizer.fit_transform(tokenized_docs)

# Aggregation: 矩陣加總
tfidf_sums = np.asarray(tfidf_matrix.sum(axis=0)).flatten()
tf_sums = np.asarray(tf_matrix.sum(axis=0)).flatten()

# 將結果轉為字典以便排序
tfidf_scores = {feature_names[i]: tfidf_sums[i] for i in range(len(feature_names))}
tf_scores = {feature_names[i]: tf_sums[i] for i in range(len(feature_names))}

# 取出前 500 名
top_500_tfidf = sorted(tfidf_scores.items(), key=lambda x: x[1], reverse=True)[:500
top_500_tf = sorted(tf_scores.items(), key=lambda x: x[1], reverse=True)[:500]
```

```
# Visualization
plot_results(top_500_tfidf, 'Top 500 Words by TF-IDF Weight (fig#1)', 'TF-IDF Summa
1, word2idx)
plot_results(top_500_tf, 'Top 500 Words by Frequency (fig#2)', 'Frequency Count',
2, word2idx)
```